



Tutorial Sheet - Artificial Intelligence

Faculty/School	ICT FACULTY
Course Code/Title	CSC 4121/Artificial Intelligence
Instructor	Dr FOTSING KUETCHE
Rank	Lecturer
E-mail address	Fotsing.fk@gmail.com

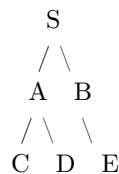
1 Introduction

1. Define *Artificial Intelligence* in your own words.
2. Differentiate between *Artificial General Intelligence (AGI)* and *Artificial Superintelligence (ASI)*.
3. List three historical events that marked the evolution of AI.
4. Explain the difference between *symbolic AI* and *connectionist AI*.
5. Name at least five major subfields of Artificial Intelligence.
6. Describe how AI can be applied in **education**, **healthcare**, and **transportation**.
7. Identify which of the following are AI applications and justify your answer:
 - Voice assistants like Siri
 - Spreadsheets performing formulas
 - Spam email filters
 - Search engines like Google
8. Given a problem such as “finding the shortest route between two cities,” classify it according to AI problem types (searching, optimization, classification, etc.) and explain your reasoning.
9. Explain why data is called the “fuel” of AI.
10. Match each AI subfield with a correct example:

Subfield	Example
Computer Vision	---
Natural Language Processing	---
Expert Systems	---
Robotics	---
Machine Learning	---

2 Searching Problems

1. What is a *search problem* in Artificial Intelligence?
2. Define *state*, *initial state*, *goal state*, and *transition model*.
3. Differentiate between *uninformed search* and *informed search*.
4. Give two examples of deterministic problems.
5. Simulate the Breadth-First Search (BFS) algorithm manually on the following tree:



What is the order of node expansion?

6. Explain how **Depth-First Search (DFS)** differs from **Breadth-First Search (BFS)** in terms of memory usage and completeness.
7. Discuss a real-world example where **deterministic optimization** is used (e.g., route planning, scheduling, or resource allocation).
8. Why is it sometimes preferable to use an *approximate solution* rather than an exact one in AI search problems?
9. Given a robot navigating a 4×4 grid with obstacles, describe how you would model this as a search problem (state representation, actions, goal).

3 Introduction to Heuristics and Meta-Heuristics

1. Define the term *heuristic* in the context of Artificial Intelligence.

2. How does a heuristic differ from an exact algorithm?
3. Explain what a *meta-heuristic* is and give two examples.
4. State two advantages and two limitations of using heuristics.
5. Why are heuristics often used in AI search problems?
6. Consider the problem of navigating a maze. Suggest one heuristic that could be used to guide the search.
7. Distinguish between **Greedy Search** and **A* Search** in terms of how they use heuristic information.
8. Identify one real-world example in which a meta-heuristic algorithm (e.g., Genetic Algorithm or Simulated Annealing) would outperform an exact search.
9. Write short pseudocode describing how a simple hill-climbing algorithm operates.
10. **Mini Project – Implementing Hill-Climbing in Python**

Write a Python program that simulates a hill-climbing algorithm for maximizing the function $f(x) = -(x - 3)^2 + 9$ within the range $[0, 6]$.

The program should start with a random value of x and iteratively move toward higher values of $f(x)$ until no further improvement is possible.

Display the number of iterations and the final optimal value.

4 Evolutionary Algorithms

1. Define an *evolutionary algorithm*.
2. List the main components of a Genetic Algorithm.
3. What are the roles of selection, crossover, and mutation in GA?
4. Explain the difference between binary and real-coded representations.
5. Identify one practical application of Genetic Algorithms in engineering or data science.
6. Outline the steps of a simple Genetic Algorithm using pseudocode.
7. Suppose you want to maximize the function $f(x) = x^2$ for $100 \leq x \leq 10$. Explain how a GA would represent and evolve candidate solutions.
8. Explain how the *fitness function* guides the search process in GA.
9. Given a small population of bit strings:

1. 10101

2. 11100
3. 01110
4. 00111

Demonstrate one iteration of selection (using roulette-wheel), single-point crossover, and mutation.

10. Explain how real-coded GAs are more suitable for continuous optimization problems.
11. Compare the performance of GAs with deterministic algorithms such as gradient descent.

13. Mini Project – Genetic Algorithm for Function Optimization

- Write a Python script to find the maximum of $f(x) = \sin(x) \times x$ in the range $[0, 10]$.
- Represent individuals as real numbers (real-coded GA).
- Implement selection, crossover, and mutation.
- Plot the evolution of the best fitness over generations.

5 Swarm Intelligence

1. Define *Swarm Intelligence*.
2. List three characteristics of swarm-based systems.
3. What is the main biological behavior that inspires ACO?
4. Explain how *pheromone trails* influence the ACO algorithm.
5. State two advantages and one limitation of PSO compared to GA.
6. Outline the steps of the Ant Colony Optimization algorithm.
7. In PSO, explain the role of each of the following parameters:
 - Inertia weight (w)
 - Cognitive coefficient (c_1)
 - Social coefficient (c_2)
8. Describe how each particle in PSO updates its position and velocity.
9. Consider a function $f(x, y) = x^2 + y^2$. Explain how PSO could be used to find its minimum.

10. Why are swarm algorithms often described as “collective problem solvers”?
11. Compare ACO and PSO in terms of their underlying mechanisms and application domains.

12. Mini Project – Implementing Particle Swarm Optimization (PSO)

- Implement a basic PSO in Python to minimize the function $f(x) = x^2 + 10\sin(x)$ within $[-10, 10]$.
- Use a small swarm of 20 particles.
- Plot or print the best fitness over 50 iterations.
- Discuss convergence behavior and final solution quality.

6 Machine Learning

1. Define *machine learning* in your own words.
2. Differentiate between *supervised* and *unsupervised* learning with examples.
3. What is a *training dataset* and how is it different from a *test dataset*?
4. State two assumptions behind the linear regression model.
5. What is *overfitting* and how can it be reduced?
6. Given the dataset below representing study hours (X) and test scores (Y):

X (hours)	Y (score)
1	45
2	50
3	60
4	70
5	80

- (a) Plot the data.
- (b) Compute the best-fit line equation $y = a + bx$ using basic linear regression logic.
- (c) Predict the score for a student who studies 6 hours.
7. Explain how *K-Nearest Neighbors (KNN)* can be used for classification.
8. Describe a real-world use case for supervised learning and one for unsupervised learning.
9. Compare *K-Means clustering* and *Hierarchical clustering* in terms of algorithmic structure.

10. Discuss the importance of data normalization before training ML models.
11. Explain why large datasets do not necessarily guarantee better model accuracy.
12. Discuss ethical concerns that may arise from using biased data in ML systems.

13. Mini Project – Linear Regression Implementation

- Write a Python program (using numpy or plain lists) to implement simple linear regression.
- Plot the regression line on the dataset from Question 6.
- Calculate and print the mean squared error (MSE).

7 Artificial Neural Networks

1. Define an *artificial neuron* and list its main components.
2. What is the mathematical function that governs a perceptron's output?
3. Explain the meaning of *weights* and *bias* in neural networks.
4. State two limitations of a single-layer perceptron.
5. How does backpropagation differ from the perceptron learning rule?
6. Consider a perceptron with:

inputs $x_1 = 1, x_2 = 0$, *weights* $w_1 = 0.4, w_2 = 0.6$, and *bias* $b = -0.5$.

Compute the weighted sum and activation output assuming a threshold activation function.

7. Implement a simple perceptron in Python that performs the logical AND operation.
8. Illustrate the step-by-step weight updates when training the perceptron on the OR gate dataset.
9. Write pseudocode describing the forward pass and backward pass of a 2-layer feedforward neural network.

10. Mini Project – Handwritten Digit Recognition (Conceptual Implementation)

- Use Python and a simple neural network framework (e.g., tensorflow or sklearn) to train a model on the MNIST dataset.

- Display the first 5 predictions with their corresponding images.
- Report training accuracy and discuss potential improvements.

8 Going Further

1. Design a hybrid approach that uses both heuristics and learning to optimize traffic light control in a city.
2. Suggest a workflow for applying AI to medical image diagnosis, highlighting data preparation, model training, and evaluation.
3. Compare the computational cost of training a neural network versus running a search-based planning algorithm.
4. Create a table summarizing when each major AI method (Search, Heuristics, GA, PSO, ML, ANN) is most appropriate.
5. Discuss how reinforcement learning could be used to improve the efficiency of industrial robots.

6. Mini Project – AI System Proposal

- Choose a real-world challenge in your community or country.
- Propose an AI-based solution, explaining:
 1. The problem context
 2. The chosen AI approach (search, learning, etc.)
 3. Required data and resources
 4. Potential benefits and ethical implications
- Write a 2–3 page proposal or present it as a short report.